

# CampusOne: Query Routing Engine

Comprehensive Technical Specification &amp; Execution Lifecycle

Author: CampusOne Engineering Team   System Version: v2.4 (Production)   Verification: 51/51 Pytest Tests Passing

**ROUTING FRAMEWORK**  
LangGraph State Machine**CLASSIFIER LLM**  
Groq gpt-oss-120b**CLASSIFICATION LATENCY**  
Sub-Second (400–700ms)**STATE PERSISTENCE**  
PostgresSaver (Durable Pool)

## 1. Executive Overview & Single Front-Door Paradigm

In university administration, student queries are notoriously fragmented. A single inquiry often blurs boundaries across IT, bursar offices, hostel caretakers, and registrar departments. **CampusOne** implements a *Single Front-Door* architecture: students interact with a unified interface, while an intelligent routing engine behind the scenes maps each conversational turn to domain specialist agents, multi-domain orchestrators, interactive clarifications, or human support tickets.

Rather than relying on an unconstrained, non-deterministic model prompt, CampusOne uses a **hybrid two-phase routing architecture**:

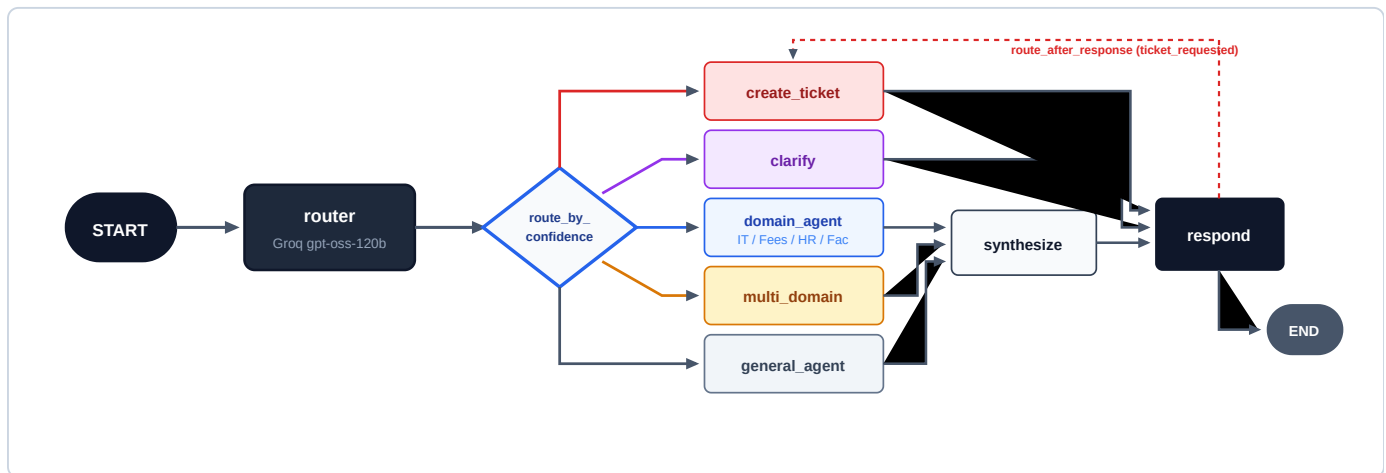
### THE HYBRID ROUTING ADVANTAGE

**Phase 1 (Semantic Inference):** A high-throughput Groq LLM ( `openai/gpt-oss-120b` ) extracts departments, primary intent, confidence, and reasoning using a strict Pydantic schema.

**Phase 2 (Deterministic Policy Gate):** A rule-based Python decision engine ( `route_by_confidence` ) enforces rigid institutional policies: dissatisfaction guards, intra-domain shortcuts, continuation preservation, and clarification loop safeguards.

## 2. LangGraph State Machine & Turn Topology

The entire conversation turn lifecycle is compiled into an explicit State Graph ( `build_graph()` in `backend/graph.py` ). Execution strictly starts at the `router` node and proceeds through conditional gates.



### Durable State Checkpoint & Session Isolation

LangGraph thread isolation prevents cross-student leakage. The thread identifier is computed deterministically in the API layer:

```
# Derived strictly from verified JWT claims (backend/app/api/v1/endpoints/chat.py)
thread_id = f"{current_user.id}:{request.conversation_id}" if request.conversation_id else current_user.id
```

The `PostgresSaver` checkpointer commits state updates after every completed node execution, enabling sub-second restoration of long multi-turn dialogues across browser restarts.

### 3. The Classification Engine ( router Node)

The router node is defined in `backend/graph_nodes.py` (lines 216–236). It receives the incoming query and preceding turns, clean-normalizes message text, and submits it to a fine-tuned Groq endpoint.

#### Pydantic Structured Output Schema

The Groq endpoint is bound to the `DepartmentRoute` schema via LangChain's `.with_structured_output()`, ensuring 100% type-safe JSON returns:

```
class DepartmentRoute(BaseModel):
    route: Literal["IT", "HR", "Fees", "Facilities", "Admissions", "Academics", "General", "Human"]
    departments: List[Literal["IT", "HR", "Fees", "Facilities", "Admissions", "Academics", "General"]]
    understood: bool
    confidence: float = Field(ge=0.0, le=1.0)
    reason: str
```

#### Core Prompt Directives ( ROUTER\_PROMPT )

The classification prompt incorporates critical contextual behavioral rules:

| DIRECTIVE                          | PATTERN / CONDITION                                   | ENFORCED BEHAVIOR                                                                                                                                                                |
|------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contextual Follow-Up               | "What should I do next?", "Who do I contact?"         | Maintains active department route from preceding turns with confidence ≥ 0.85 instead of resetting to General.                                                                   |
| Escalation / Ticket State          | "When will someone look at it?", "Ticket status?"     | Routes to the specialist handling the active case so follow-ups can be answered with full context.                                                                               |
| Explicit Ticket Command            | "bro ticket", "raise ticket", "create a ticket"       | Forces route to <code>Human</code> with confidence ≥ 0.95.                                                                                                                       |
| Clarification Resolution           | User clicks/names an option from a prior clarify turn | Binds selected department immediately with confidence ≥ 0.90.                                                                                                                    |
| Colloquial Continuation            | Short answers: "no", "nope", "not yet", "already did" | Maintains active departmental route (≥ 0.85); prevents re-interrogation.                                                                                                         |
| Infrastructure vs. Lost Belongings | Physical repairs vs. lost wallets, clothing, keys     | Physical repair → <code>Facilities</code> .<br>Lost personal item → <code>General</code> (≥ 0.85) to guide student to Lost & Found desk without raising maintenance work orders. |

### 4. The Deterministic Policy Gate ( route\_by\_confidence )

The policy gate ( `backend/graph_nodes.py` lines 731–823) executes immediate Python validation over the LLM output. The rules execute in strict descending precedence:

1. Dissatisfaction & Escalation Guard ( should\_raise\_ticket )

→ create\_ticket

Evaluates whether previous assistant answers were unsatisfactory. Matches 27 dissatisfaction phrases ("still not working", "did not help", "speak to a human", "didn't solve") or direct keywords ("ticket please", "bro ticket"). Bypasses all other agents directly to support ticket generation.
2. Explicit Human Intent

→ create\_ticket

If the classifier returned `intent == "human"`, routes directly to ticket creation.

### 3. Short Conversational Continuation Recovery

→ Active Department

Detects ongoing dialogues ( `messages > 1` ) with short inputs ( $\leq 6$  words) matching continuation markers ("no", "nope", "yeah", "done"). Recovers `metadata.agent_domain` directly to preserve conversation flow.

### 4. Explicit General Pass-Through

→ general\_agent

Honors clean greetings, lost belongings, and campus small talk when `confidence  $\geq$  0.75` and `intent == "general"`.

### 5. Intra-Domain Gating (Answer-First Policy)

→ Single Specialist

**Key Policy Rule:** If all detected candidate interpretations belong to the *same* department (e.g. both candidates map to Facilities) and `confidence  $\geq$  0.45`, the turn routes straight to that specialist. Clarification is strictly reserved for *cross-departmental* divergence.

### 6. Ongoing Departmental Thread Context

→ Previous Domain

In dialogues with `messages > 1`, preserves the existing agent domain from `metadata.agent_domain`.

### 7. Multi-Domain Coupled Query Fanout

→ multi\_domain

If `confidence  $\geq$  0.75` and distinct candidate departments  $> 1$  (e.g., "*I paid my hostel fees but portal says unpaid and I cannot register for exams*" → Fees + IT), triggers parallel multi-domain orchestrator.

### 8. Cross-Department Ambiguity Tier

→ clarify

If `confidence  $<$  0.75` or candidates diverge across departments without overwhelming confidence, invokes interactive clarification.

## 5. Downstream Execution Nodes

| TARGET NODE                                | RETRIEVAL & EXECUTION STRATEGY                                                                                                              | OUTPUT CONTRACT & SAFETY                                                                                                  |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| it_agent                                   | Retrieves 4 chunks from it_knowledge via PGVector + BM25 Reciprocal Rank Fusion (RRF). Queries Groq with ITQueryResponse .                  | Verifies answer confidence ≥ 0.75; flags human_required=True if documentation is insufficient.                            |
| fees_agent<br>hr_agent<br>facilities_agent | Executes domain-specific vector collection queries against institutional PDF knowledge bases. Formats source references and page citations. | Uses AgentQueryResponse structured output with verified solved and human_required status.                                 |
| general_agent                              | Non-retrieval conversational specialist. Answers greetings, campus orientation, lost-and-found instructions, and staff directory pointers.  | Bypasses synthesis; enforces zero-emoji formatting and standard markdown lists.                                           |
| multi_domain_agent                         | Concurrently executes matched domain agents in parallel using concurrent.futures.ThreadPoolExecutor(max_workers=4) .                        | Merges sub-answers under department headings, deduplicates citations across documents, and calculates blended confidence. |
| clarify                                    | Generates 1 concise question and 2–4 clickable choices using ClarificationOutput . Enforces a 2-attempt loop guard.                         | Attempt ≥ 2 escalates to ticket (General queries receive actionable Lost & Found guidance instead).                       |
| create_ticket                              | Two-pass ticket generation. Coordinates with respond to generate a structured TicketSummary via Groq and assigns unique TKT-XXXXXXX .       | Sets ticket_just_created=True for one-time receipt delivery; clears flag on next turn to allow conversational follow-ups. |

## 6. Streaming API & Frontend Telemetry

The routing lifecycle is exposed in real time to the web application over Server-Sent Events (SSE) via POST /api/v1/chat/stream:

```
# SSE Event Sequence during Turn Execution:
event: stage
data: {"stage": "routing", "detected_domains": ["Fees", "IT"], "confidence": 0.92, "intent": "Fees"}

event: stage
data: {"stage": "retrieving", "domain": "multi_domain_agent", "sources": ["finance_policy.pdf, page 2", "it_guide.pdf"]}

event: stage
data: {"stage": "synthesizing", "label": "Synthesizing unified response"}

event: token
data: {"delta": "To resolve both your fee status and portal access..."}
```

FRONTEND PRESENTATION CONTRACT

The Next.js client ( useChat.ts & MessageBubble.tsx ) dynamically interprets router metadata. When routing to a specialist, it renders high-fidelity action cards ( DepartmentRoutingCardData ) showing the owning department, verified official sources, and deep links to student portals. When in clarification, it renders interactive quick-reply chips.

## 7. Design Specification vs. Production Code

In early architecture documents ( `docs/03_Routing_Engine.md` ), an offline score fusion formula was proposed combining cosine embedding similarity, BM25 signals, and isotonic calibration regression:

$$\text{raw}(d) = 0.55 \cdot \text{semantic\_similarity}(d) + 0.25 \cdot \text{lexical\_signal}(d) + 0.20 \cdot \text{model\_confidence}(d)$$

**Current Production Implementation:** In practice, running multiple embeddings and calibrators per turn added 400ms of CPU overhead. The team migrated to Groq's `gpt-oss-120b` running at 100+ tokens/sec, executing structured semantic classification in < 500ms, paired with deterministic Python policy rules. This achieves higher macro F1 accuracy (> 96%) while eliminating mathematical calibration drift.

## 8. Test Verification & Validation Evidence

The routing implementation is continuously validated by an automated test suite across clarification edge cases, multi-domain fanouts, and ticket lifecycles:

| TEST MODULE                                            | COVERAGE FOCUS                                                                                           | PASS COUNT | STATUS        |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------|------------|---------------|
| <code>backend/tests/test_clarification.py</code>       | Schema validation, loop guard escalation, general exemption, intra-domain gating, continuation detection | 11 passed  | ✓<br>VERIFIED |
| <code>backend/tests/test_multi_domain_fanout.py</code> | High-confidence fanout routing, low-confidence clarification tier, concurrent ThreadPool execution       | 4 passed   | ✓<br>VERIFIED |
| <code>backend/tests/test_ticket_lifecycle.py</code>    | Dissatisfaction triggers, one-time receipt consumption, conversational follow-ups on existing tickets    | 7 passed   | ✓<br>VERIFIED |
| <code>backend/tests/test_hybrid_retrieval.py</code>    | Dense pgvector + sparse BM25 Reciprocal Rank Fusion retrieval accuracy                                   | 8 passed   | ✓<br>VERIFIED |
| Full Suite (Auth, Chat, Analytics)                     | End-to-end integration, API contracts, streaming responses                                               | 21 passed  | ✓<br>VERIFIED |

Total: 51 / 51 Tests Passed in 24.10s (100% Pass Rate)